Problem 1: SCC and Reversal1. Graph Reversal Algorithm $O(V + E)$To compute $rev(G)$, we iterate through the adjacency list of G and reverse every edge.

In [1]:
```python
def reverse_graph(adj):
    # Initialize new adjacency list for rev(G)
    rev_adj = {u: [] for u in adj}

    # Traverse each vertex and its neighbors
    for u in adj:
        for v in adj[u]:
            # Reverse the edge: (u, v) becomes (v, u)
            rev_adj[v].append(u)

    return rev_adj

# Test the reversal
g = {'A': ['B', 'C'], 'B': ['D'], 'C': [], 'D': []}
print(f"Original G: {g}")
print(f"Reversed G: {reverse_graph(g)}")
```

```
Original G: {'A': ['B', 'C'], 'B': ['D'], 'C': [], 'D': []}
Reversed G: {'A': [], 'B': ['A'], 'C': ['A'], 'D': ['B']}
```

Complexity: We visit each vertex once and each edge once, so the time complexity is $O(V + E)$.2. Proof: $scc(G)$ is AcyclicSuppose $scc(G)$ contains a cycle $C_1 \to C_2 \to \dots \to C_k \to C_1$.This implies there is a path from any vertex in C_1 to any vertex in C_i, and a path back from C_i to C_1.By the definition of Strongly Connected Components, if u can reach v and v can reach u, they must belong to the same component.Thus, $C_1, \dots, C_k$ are not distinct; they are one single SCC. This contradicts the definition of the component graph. Therefore, $scc(G)$ must be a Directed Acyclic Graph (DAG).3. Proof: $scc(rev(G)) = rev(scc(G))$Nodes: Two vertices u, v are in the same SCC in G if there are paths $u \leadsto v$ and $v \leadsto u$. In $rev(G)$, these paths are simply flipped, so u, v remain in the same SCC. Thus, the nodes (components) of $scc(G)$ and $scc(rev(G))$ are identical.Edges: If there is an edge $C_i \to C_j$ in $scc(G)$, it means there is an edge (u, v) in G ($u \in C_i, v \in C_j$). In $rev(G)$, this becomes (v, u), creating an edge $C_j \to C_i$ in $scc(rev(G))$. This is exactly the definition of $rev(scc(G))$.4. Reachability Property($\implies$): If $u \leadsto v$ in G, the path traverses a sequence of SCCs. This sequence directly forms a path from $S(u)$ to $S(v)$ in $scc(G)$.($\impliedby$): If $S(u) \leadsto S(v)$ in $scc(G)$, there are "bridge" edges

between components in G. Since any vertex inside an SCC can reach any other vertex in that same SCC, we can connect these bridges to form a path $u \leadsto v$ in G.

Problem 2: Euler Tour1. Necessary and Sufficient ConditionA strongly connected graph G has an Euler tour if and only if $in\text{-}degree(v) = out\text{-}degree(v)$ for every vertex $v \in V$.Intuition: Every time we enter a vertex via an edge, we must be able to leave it via a different edge to eventually complete the cycle.2. $O(E)$ Algorithm Implementation (Hierholzer's)The algorithm finds a cycle, then merges it with other edge-disjoint cycles.

In [2]:
```
def find_euler_tour(adj_list, start_node):
    # Use a copy to avoid destroying original graph
    graph = {u: list(v) for u, v in adj_list.items()}
    stack = [start_node]
    tour = []

    while stack:
        u = stack[-1]
        if graph[u]:
            v = graph[u].pop()
            stack.append(v)
        else:
            tour.append(stack.pop())

    return tour[::-1]

# Example: A simple cycle A->B->C->A
example_g = {'A': ['B'], 'B': ['C'], 'C': ['A']}
print(f"Euler Tour: {find_euler_tour(example_g, 'A')}")
```

```
Euler Tour: ['A', 'B', 'C', 'A']
```

Problem 3: Topological SearchCourse dependencies: $\{A \to B, A \to C, B \to C, B \to D, C \to E, D \to E, D \to F, G \to F, G \to E\}$

In [3]:
```
from collections import deque

def topological_sort(nodes, edges):
    in_degree = {u: 0 for u in nodes}
    adj = {u: [] for u in nodes}
    for u, v in edges:
```

```python
        adj[u].append(v)
        in_degree[v] += 1

    # Start with nodes that have no prerequisites
    queue = deque(sorted([u for u in nodes if in_degree[u] == 0]))
    result = []

    while queue:
        u = queue.popleft()
        result.append(u)
        for v in adj[u]:
            in_degree[v] -= 1
            if in_degree[v] == 0:
                queue.append(v)
    return result

nodes = ['A', 'B', 'C', 'D', 'E', 'F', 'G']
edges = [('A','B'), ('A','C'), ('B','C'), ('B','D'), ('C','E'), ('D','E'), ('D','F'), ('G','F'), ('G','E')]

print(f"Sorting Result: {topological_sort(nodes, edges)}")
```

```
Sorting Result: ['A', 'G', 'B', 'C', 'D', 'E', 'F']
```

2. Manual SortingsBased on the dependency constraints, here are two valid orders:Starting from A: A, B, D, C, G, E, F (Note: G can be inserted anywhere as long as it's before E and F).Starting from G: G, A, B, C, D, F, E (Wait for D and G to finish before F).

In [ ]:

```python

```

In [ ]:

```python

```